DOCUMENTATION : SPLINE QUADRATIQUE

Calcul des Z_i

```python
def calculer_zi(x_points, y_points, z0=0.0):
    n = len(x_points) - 1
    z = np.zeros(n + 1)
    z[0] = z0
    for i in range(n):
        hi = x_points[i + 1] - x_points[i]
        z[i + 1] = 2.0 * (y_points[i + 1] - y_points[i]) / hi - z[i]
    return z
```

Cette fonction calcule $z_i = S'(x_i)$

Quand tu fais une **spline quadratique**, tu construis des morceaux de paraboles entre chaque point. $S'(x_i)$ garantit que les paraboles soient collées et lisses.

La formule du cours utilisée est : $z_{i+1} = 2\,\frac{y_{i+1} - y_i}{x_{i+1} - x_i} - z_i$

Position de x

```python
def position(x, x_points):
    n = len(x_points) - 1
    if x < x_points[0] or x > x_points[n]:
        raise ValueError(
            f"x = {x} hors de l'intervalle [{x_points[0]}, {x_points[n]}]"
        )
    i = 0
    while i < n - 1 and x > x_points[i + 1]:
        i += 1
    return i
```

Le but de cette fonction est de choisir le bon morceau de spline pour calculer S(x)

Evaluation des points

```python
def evaluer_spline(x, x_points, y_points, z):
    if isinstance(x, (list, np.ndarray)):
        return np.array([evaluer_spline(xi, x_points, y_points, z) for xi in x])
    i  = position(x, x_points)
    hi = x_points[i + 1] - x_points[i]
    dx = x - x_points[i]
    return (z[i + 1] - z[i]) / (2.0 * hi) * dx**2 + z[i] * dx + y_points[i]
```

L'évaluation de la spline consiste à déterminer pour un x € $[x_i ; x_{i+1}]$, la valeur S(x) = y tout en choisissant le bon intervalle. Cela permet de tracer les paraboles avec plus précision.

Calcul symbolique : sympy

```
136  def _to_sym(val):
137      """Entier SymPy si val est entier, sinon Float 15 chiffres."""
138      v = float(val)
139      return sp.Integer(int(v)) if v == int(v) else sp.Float(v, 15)
```

Cette fonction convertie les entiers et réels pythons en entiers er réels sympy.

```
142  def spline_sympy(x_points, y_points, z0=0):
```

Cette fonction fait toutes les étapes de spline (calcul des z_i, vérification de la position des x et l'évaluation spline (construction de S(x))).

```
161      x = sp.Symbol('x')
```

On crée la variable mathématique x

```
164      xr = [_to_sym(xi) for xi in x_points]
165      yr = [_to_sym(yi) for yi in y_points]
```

On convertit les données d'entrée en objets SymPy.

```
168      z = [None] * (n + 1)
169      z[0] = _to_sym(z0)
170      for i in range(n):
171          hi      = xr[i + 1] - xr[i]
172          z[i + 1] = sp.Integer(2) * (yr[i + 1] - yr[i]) / hi - z[i]
```

L 168 à L 172 : ce bloc fait le même travail que la fonction **calculer_zi()** ci-dessus.

L 168 et L 169, initialisent avec z_0 *la condition initiale*

```
175      morceaux = []
176      for i in range(n):
177          hi = xr[i + 1] - xr[i]
178          xi = xr[i]
179          yi = yr[i]
180
181          Si = sp.expand(
182              (z[i + 1] - z[i]) / (2 * hi) * (x - xi)**2
183              + z[i] * (x - xi)
184              + yi
185          )
186          morceaux.append((Si, xr[i], xr[i + 1]))
```

Cette partie construit directement les $S_i(x)$ sur $[x_i ; x_{i+1}]$ comme donné au cours sur l'image ci-dessous. L 186, on garde S_i et son intervalle.

$$S_i(x) = \frac{1}{2}\frac{z_{i+1}-z_i}{x_{i+1}-x_i}(x-x_i)^2 + z_i(x-x_i) + y_i$$

```python
    pieces = []
    for i, (Si, xi, xi1) in enumerate(morceaux):
        cond = ((x >= xi) & (x < xi1)) if i < n - 1 else ((x >= xi) & (x <= xi1))
        pieces.append((Si, cond))
    S_piecewise = sp.Piecewise(*pieces)
```

Ici, on construit la fonction $S(x)$ globale comme sur l'image ci-dessous :

$$S(x) = \begin{cases} S_0(x) = a_0 x^2 + b_0 x + c_0 & \text{si } x \in [x_0, x_1] \\ S_1(x) = a_1 x^2 + b_1 x + c_1 & \text{si } x \in [x_1, x_2] \\ \vdots \\ S_i(x) = a_i x^2 + b_i x + c_i & \text{si } x \in [x_i, x_{i+1}] \\ \vdots \\ S_{n-1}(x) = a_{n-1} x^2 + b_{n-1} x + c_{n-1} & \text{si } x \in [x_{n-1}, x_n] \end{cases}$$

<u>Calcul numérique : SciPy</u> :

```python
202 def spline_scipy(x_points, y_points, z0=0.0):
222     x_arr = np.array(x_points, dtype=float)
223     y_arr = np.array(y_points, dtype=float)
224     n = len(x_arr) - 1
225
226     z = calculer_zi(x_arr, y_arr, float(z0))
227
228     a_coef = np.array([
229         (z[i + 1] - z[i]) / (2.0 * (x_arr[i + 1] - x_arr[i]))
230         for i in range(n)
231     ])
232
233     # PPoly forme centrée : [a_i, z_i, y_i]
234     coeffs = np.array([a_coef, z[:-1], y_arr[:-1]])
235     return PPoly(coeffs, x_arr)
```

L 222 – L 224 : préparation des données ;

L 226 : calcul des z_i sur les intervalles.

L 228 – L 231 : on calcul les coefficients directeurs de chaque parabole. La forme des paraboles devient donc : $S_i(x) = a_i(x - x_i)^2 + z_i(x - x_i) + y_i$

L 234 : on construit les coefficients a_i, z_i et y_i en les stockant dans un tableau.

L 235 : C'est la création de la Spline. PPoly (Piecewise Polynomial) contient : tous les moreaux, les intervalles et les coefficients.

<u>DIFFÉRENCES CLÉS ENTRE LES TROIS APPROCHES</u>

	NumPy	SymPy	SciPy
Le type de résultat	Nombre	Formule	Fonction
Le niveau d'abstraction	Bas niveau	Mathématique	Haut niveau
L'utilisation	Apprentissage	Théorie	Pratique